

Debugging and Troubleshooting

General Problem Reporting

When reporting a problem, please always include:

- `nvidia-bug-report.log.gz` - produced by `nvidia-bug-report.sh`
- Full output of `dcgmi -v`
- Relevant and/or requested logs, below

Logging

This topic discusses various ways DCGM can be configured in order to produce detailed logs.

Use [nv-hostengine](#) for the complete host-engine logging and environment reference, and [dcgmi diag](#) for diagnostic logging options and defaults. The examples below focus on collecting evidence for a problem report.

Note

When the packaged service owns the host engine, inspect it with `journalctl --unit=nvidia-dcgm.service --boot` instead of starting a second server process merely to see console output. See [the journal explanation](#) for filtering and access considerations.

Enable Logging Using Standalone Hostengine

When launching `nv-hostengine`:

- Add the `-f /path/to/log` parameter to specify where to write the log
- Add the `--log-level DEBUG` parameter to specify DEBUG logging

This example will collect debug logs from the standalone hostengine for the duration of its lifetime. The log file will be written to `/tmp/nv-hostengine.log`. Example:

The process that opens the log must have permission to traverse the parent directory and create or write the file. See [the Linux process-credentials note](#) when a command works as root but not under the packaged service account.

```
% sudo nv-hostengine -f /tmp/nv-hostengine.log --log-level DEBUG
```

Enable Logging Using Embedded Hostengine

When using an embedded hostengine, if running as the `root` user or other privileged user, first change to that user using the appropriate command, i.e., `sudo(8)`. While still in the same session that you ran `sudo`:

Note

Exported variables are inherited by commands subsequently launched from that shell; they do not modify another login shell or an existing systemd service. This is why the exports and embedded command must run in the same privileged shell; see [process environments](#) and [administrative privileges](#).

- Use `export __DCGM_DBG_FILE=/path/to/log` to specify where to write the log
- Use `export __DCGM_DBG_LVL=DEBUG` to specify DEBUG logging
- Use `env | grep __DCGM_DBG` to confirm the variables are set
- Run the desired command

This example will collect debug logs from the embedded host engine while running the short diagnostic. The log file will be written to `/tmp/embedded.log`. Example:

```
% sudo -i
(prompts for password)
# export __DCGM_DBG_FILE=/tmp/embedded.log
# export __DCGM_DBG_LVL=DEBUG
# env | grep __DCGM_DBG
(output)
__DCGM_DBG_FILE=/tmp/embedded.log
__DCGM_DBG_LVL=DEBUG
# dcgmi diag -r short
...
```

Enable Diagnostic Logging

The diagnostic produces additional useful logging. When running the diagnostic through `dcgmi`:

- Add the `--debugLogFile /path/to/log` parameter to specify where to write the log
- Add the `-d DEBUG` parameter to specify DEBUG logging

This example will collect debug logs from the short diagnostic. The log file will be written to `/tmp/nvvs.log`. Example:

```
% dcgmi diag -r short --debugLogFile /tmp/nvvs.log -d DEBUG
```

Enable NVML Logging

In some cases, NVIDIA engineers may request NVML logs to aid in debugging.

If running as the `root` user or other privileged user, first change to that user using the appropriate command, i.e., `sudo(8)`. While still in the same session that you ran `sudo`:

- Use `export __NVML_DBG_FILE=/path/to/log` to specify where to write the log
- Use `export __NVML_DBG_LVL=DEBUG` to specify DEBUG logging
- Use `env | grep __NVML_DBG` to confirm the variables are set
- While still in the same session, add any other necessary environment variables (i.e., if you are running an embedded host engine)
- Run the desired command

Note

If using the standalone hostengine, a separate `__NVML_DBG_FILE` should be specified for the hostengine and the desired command. See the example that follows.

This example will collect NVML logs and debug logs from a standalone hostengine, as well as NVML and debug logs from the long diagnostic. The NVML logs for the hostengine will be written to

`/tmp/hostengine.nvml.log`, and the NVML logs for the diagnostic will be written to `/tmp/nvvs.nvml.log`. Example:

```
% sudo -i
(prompts for password)
# export __NVML_DBG_FILE=/tmp/hostengine.nvml.log
# export __NVML_DBG_LVL=DEBUG
# env | grep __NVML_DBG
(output)
__NVML_DBG_FILE=/tmp/hostengine.nvml.log
__NVML_DBG_LVL=DEBUG
# nv-hostengine -f /tmp/nv-hostengine.log --log-level DEBUG
# export __NVML_DBG_FILE=/tmp/nvvs.nvml.log
# env | grep __NVML_DBG
(output)
__NVML_DBG_FILE=/tmp/nvvs.nvml.log
__NVML_DBG_LVL=DEBUG
# dcgmi diag -r long --debugLogFile /tmp/nvvs.log -d DEBUG
...
```

Troubleshooting

Note

If a DCGM process ends unexpectedly, capture its status before running another shell command and compare it with the service journal. A signal can terminate a process without a normal application return; see [signal\(7\)](#) and the [Bash exit-status rules](#).

Host Engine Environment Variables Affecting Hang Detection

The `nv-hostengine` program accepts environment variables that control hang detection. These values must be present in the environment of the process that starts the host engine; see [process environments](#).

The following environment variables affect hang detection in the hostengine and in hostengine modules:

<code>DCGM_HANGDETECT_DISABLE</code>	When set, disables the hang detection system in the hostengine and in hostengine modules. Hang detection is enabled by default and monitors select capabilities for hangs. This does not change the response to hangs in the diagnostic. See Environment in Diagnostics for more information.
<code>DCGM_HANGDETECT_EXPIRY_SEC</code>	Sets the time period (in seconds) after which unresponsive threads/processes may be considered hung.
<code>DCGM_HANGDETECT_TERMINATE</code>	When set, attempts to terminate the hostengine process if a hang is detected. By default, the hostengine will attempt to log a message and continue, allowing the reported hang to continue. This does not change the response to hangs in DCGM modules, which are

